LaunchDarkly

MORE SIGNAL PER CALL

How many metrics can one LLM judge score before it lies?



Pick a number: **10? 20? 40?**

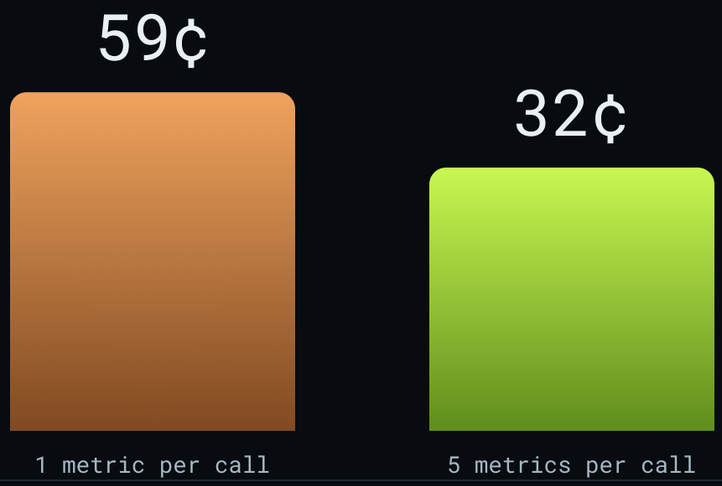
Scarlett Attensil · LaunchDarkly

Conf42 Platform Engineering 2026

01 · THE COST STRUCTURE

A thousand metrics cost 59¢ separately. Packed together, they cost 32¢.





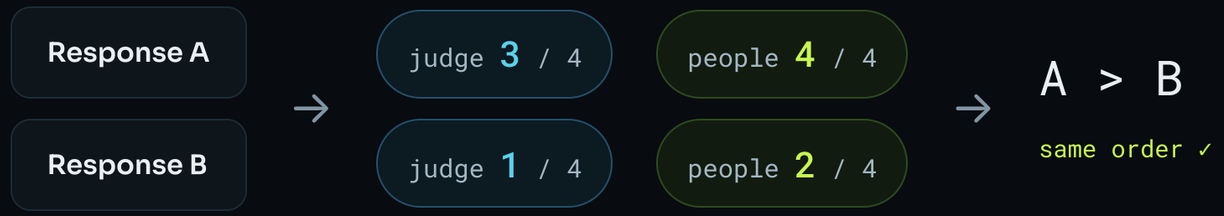
46% cheaper across 3,599 completed calls. The judge reads the response once and reasons once.

> **Only need labels?** Jev answers typed questions without generating text. Try a decision model before you optimize a generative judge.

Deußer, Sparrenberg & Sifa (2026) · arXiv:2609.37647

02 · THE DATA AND THE SCORE

Score each response. Then check the order.



HelpSteer2: 10,681 prompts · two responses · five human-scored metrics · each scored 0 to 4 by at least three people. The judge scores one response per call, never side by side.



03 · THE FIRST EXPERIMENT

Change the packing. Keep everything else.



5 calls

ONE METRIC IN EACH

1 call

ALL FIVE METRICS TOGETHER

```
"verbosity": {
  "quote": "verbatim evidence",
  "explanation": "one or two sentences",
  "score": 3
}
```

> Same prompt, responses, rubric and output budget per metric.

src/packing/budget.py

```
    headroom = REASONING_HEADROOM_TOKENS if reasoning else 0
    content = ENVELOPE_OVERHEAD_TOKENS + PER_CRITERION_OUTPUT_TOKENS * group_size
```

04 · BUILDING THE CALL

One prompt template. Any number of metrics.



src/packing/criteria.py

```
    def render(self) -> str:
        """Rubric block for the judge prompt."""
        lines = [f"### {self.label} ({self.key})", self.description,
"", "Scale:"]
        for value in sorted(self.anchors):
            lines.append(f"  {value} = {self.anchors[value]}")
        return "\n".join(lines)
```

Every metric renders the same way: name, definition, and a description for each score.

src/packing/prompt.py

```
## Rubric ({len(group)} {plural})

{_rubric_block(group)}

## Task

Score the response on each of the {len(group)} {plural} above,
independently. \
For each one, give a short verbatim quote as evidence, one or two
sentences of \
reasoning, and then the integer score.
```

The rubric block grows. The instructions never change.

05 · WHY THIS IS NOT OBVIOUS

Two hypotheses, both plausible



Packing hurts

Divided attention. One pass, now five judgements.

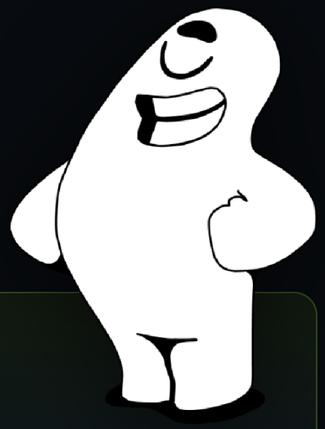
Packing helps

Criteria in context. Verbose *relative to* helpful.

> **Written down in advance:** maybe position in the list matters more than how many metrics share the call.

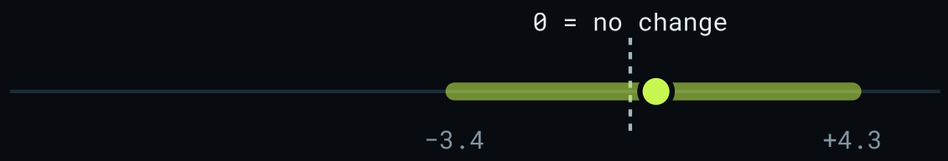
06 · THE RESULT

At five metrics, no measurable loss



+0.5 pts
ACCURACY CHANGE

46% less
COST PER METRIC



No measurable loss. 95% CI −3.4 to +4.3 points across 144 paired prompts.

07 · RELIABILITY CHECK

Packing did not increase ties

48.00%
ONE METRIC PER CALL

47.96%
FIVE METRICS PER CALL

> The judge was no more indecisive when the metrics shared a call.

08 · BEYOND THE HUMAN LABELS

Eight ways I broke an answer



duplicate verbosity ↑

- Data Mining - Data Mining

filler conciseness ↓

It should be borne in mind that here is a haiku about a banana

agreement grammar ↓

I must inform you that I does not accept it

depunct punctuation ↓

capture the texture detail and quality

shuffle coherence ↓

These solutions are found by... x = -a ... The cubic equation you provided...

truncate completeness ↓

With a wagging tail, / He greets us every morning, [end]

drop_answer helpfulness ↓

[deleted: To fix this issue, you'll need to install...] You can do this by running...

flatten formatting ↓

a question for you: **Question 1:** ... **Score: 8/10** all on one line

numbers correctness ↓ · separate check

movies which came out between 4000 and 2008

Real edits from the retained dataset. Each targets one metric with a known direction.

09 · BUILDING THE TEST SET

Every corruption declares what it should break

src/packing/corrupt.py

```
    Corruption(
        key="duplicate",
        targeted_criterion="verbosity",
        direction=+1,
        default_severity=3,
        description="Repeat sentences verbatim; adds no new claims.",
        _apply=duplicate_sentences,
    ),
```

Target metric, expected direction, and how hard to push.

> **Every generated pair records:** operation + target + direction + seed.

src/packing/ladder_dataset.py

```
    side_seed = partition_seed(f"lad-side:{base.item_id}:{op_key}", 0,
0, seed)
    corrupt_is_b = random.Random(side_seed).random() < 0.5
    first, second = (
        (clean, result.text) if corrupt_is_b else (result.text, clean)
    )
```

A coin flip decides which side is broken, so a judge that favors the first answer gains nothing.

10 · THE SYNTHETIC EXPERIMENT

One metric is measured. The other 39 add load.





src/packing/runner.py

```
        for nominal_n in levels:
            group = (target, *companions[: nominal_n - 1])
```

- **1 measured target** has a known direction.
- **39 load-only metrics** make the call larger.
- The **target stays first** at every rung.

> **No new human labels.** Eight operations contribute 58 prompts each. The outcome is recovery of the known edit.

11 · THE EXTENSION

At forty metrics, recovery falls



-6.3 pts

RECOVERY AT 40 METRICS VS 1

-10.7 to -1.9

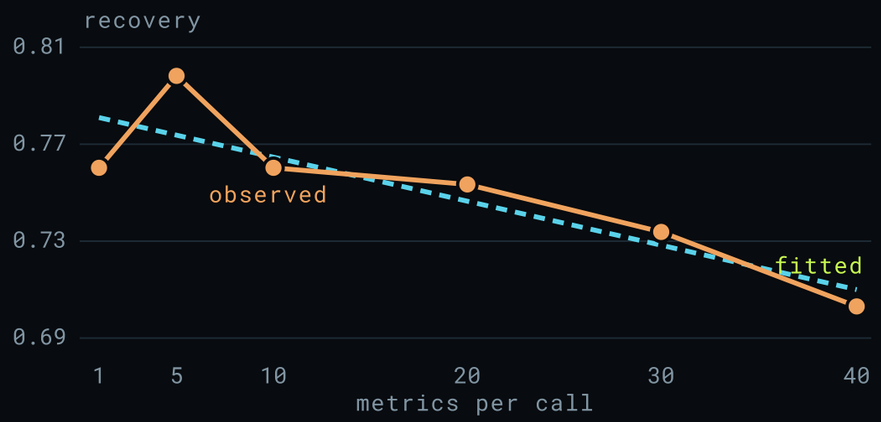
95% CONFIDENCE INTERVAL



Target stayed first. 7,653 calls · p = 0.0054. Same scale as the five-metric result. This range never reaches zero.

12 · WHERE DOES IT BREAK

It bends.
I couldn’t find a cliff.





~1%

LOWER ODDS OF CATCHING THE EDIT PER METRIC ADDED

> **Zoomed y-axis: 69% to 81% recovery.** The bump at 5 metrics is noise. A straight line fits every step.

13 · UNDER THE AVERAGE

Some breaks got much harder to spot



67% → 30%

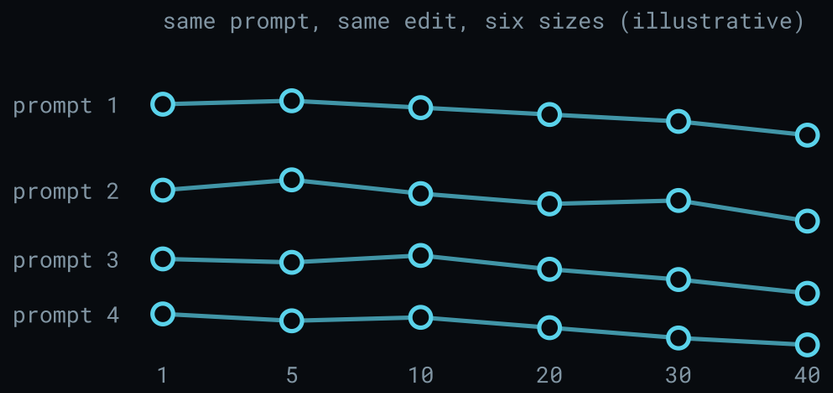
REPEATED SENTENCES CAUGHT, 1 VS 40 METRICS

Repetition, grammar and punctuation are small, local edits. Cut-off endings and wrong numbers got easier to catch.

Paired change in recovery, 1 vs 40 metrics. The three declines survive Holm correction.

14 · REPEATED MEASURES

Every prompt is its own control



src/packing/analyze.py

```
    paired = per_unit[[reference_size, comparison_size]].dropna()
    if paired.empty:
        return None
    diffs = (paired[comparison_size] -
paired[reference_size]).rename("d")
```

Each prompt is compared with itself at 1 and 40 metrics. Then each corruption type gets equal weight.

What I found

5	No measurable loss. **46% cheaper.**
40	Recovery **6.3 points lower**, target still first.
1–40	A steady bend. **No cliff.** Small local edits suffer first.



> One model (gpt-5-mini), one dataset, one rubric style. The harness is built to rerun on yours.

16 · TAKEAWAY

If you run judges in production

- **Pack a small rubric.** Five is a safe start.
- **Retest as it grows**, per metric type.
- **Describe every score** on the scale.
- **Report ties** next to accuracy.



src/packing/cli.py

```
    results = compare(
        claims, expected,
        tolerance=FLOAT_TOLERANCE if tolerance is None else
    )
    failures = [r for r in results if not r.ok]
```

```
$ uv run packing reproduce
PASS: 40 published values recomputed from retained artifacts
```

Reproduce it. Data, code and every call are in the repo.

github.com/sattensil/multi-metric-judge

Scarlett Attensil · LaunchDarkly